Table des matières

Cours 00 — Mettre en place l'environnement (DuckDB, puis PostgreSQL) 1
Partie 1 — Un entrepôt et sa base : AdventureWorks DW (à lire avant de taper) 2
Partie 2 — Monter AdventureWorks DW dans DuckDB 2
Partie 3 — Vérifier que l'entrepôt est prêt 4
Partie 4 — PostgreSQL et AdventureWorks OLTP (le second terrain) 5
Dépannage 8
Annexe — Le schéma en étoile d'AdventureWorks DW (référence) 8
Et ensuite ? 9

Cours 00 — Mettre en place l'environnement (DuckDB, puis PostgreSQL)

Module d'installation — Formation SQL avancé

Ce module se fait une seule fois. À la fin, vous aurez les deux bases de la formation sur votre poste, et vous ne le referez plus : tous les cours s'appuieront dessus.

La formation est analytique d'abord : on travaille en priorité dans un entrepôt de données (OLAP, pour *Online Analytical Processing*), le monde des tableaux de bord et de la modélisation dimensionnelle, celui du métier visé. Mais elle enseigne un langage, pas une plateforme, et un langage ne se comprend qu'en le voyant tourner sur deux terrains. Elle utilise donc deux moteurs, dans cet ordre.

- DuckDB, un moteur analytique local. C'est le moteur de la voie entrepôt, celui sur lequel la théorie s'illustre. Requis dès le Cours 01. Il est déjà installé : c'est l'étape 3 du guide d'installation du parcours, `Data_Domain/00_Installation/Guide_installation.md`.
- PostgreSQL, un moteur transactionnel (OLTP, pour *Online Transaction Processing*). C'est le second terrain. À partir du Cours 02, chaque module a deux laboratoires, un par moteur, qui posent les mêmes questions sur deux schémas opposés. Son installation est à l'étape 7 du même guide. Vous pouvez la faire tout de suite ou juste avant le Cours 02, mais pas plus tard : sans elle, vous perdez la moitié de chaque cours.

Rien à installer ici. Ce module ne pose aucun logiciel : le socle du poste vit dans le guide d'installation du parcours, qui sert aussi aux autres formations. Ce qui se fait ici, et nulle part ailleurs, c'est de charger les données des deux terrains.

Et Snowflake. La plateforme Snowflake n'est pas le moteur de cette formation, et le lecteur n'a aucun compte à ouvrir. Là où une tournure lui est propre et change le geste, un encadré « Portabilité » le dit, pour que le SQL appris ici se transporte. La plateforme elle-même est le sujet de la formation *Snowflake (SnowPro Core)*.

© 2026 iAlexMG. Tous droits réservés.

Partie 1 — Un entrepôt et sa base : AdventureWorks DW (à lire avant de taper)

Pour apprendre le SQL, il faut des données sur lesquelles s'exercer. On utilise AdventureWorks, un jeu de données d'exemple que Microsoft publie gratuitement. Concrètement, ce n'est rien de plus qu'un ensemble de fichiers, un par table, remplis de lignes fictives, qu'on télécharge puis qu'on importe dans notre moteur. C'est tout ce que fait la Partie 2 : charger ces fichiers dans une base vide.

Ces données décrivent une entreprise fictive, Adventure Works Cycles, qui fabrique et vend des vélos partout dans le monde. De quoi travailler sur du concret.

Fait important : AdventureWorks existe en deux versions, une par monde, et on démarre par la version entrepôt, le « DW », pour *Data Warehouse*.

Version	**Moteur**	**Comment c'est rangé**	**À quoi ça sert**
DW (*Data Warehouse*), notre base de référence	DuckDB	en étoile : une grosse table de faits (les ventes) entourée de tables de dimensions (produit, date, client)	analyser l'entreprise, croiser les ventes par produit, date, client, région
OLTP (*Online Transaction Processing*), le second terrain	PostgreSQL	normalisée : 68 tables découpées pour éviter les doublons	faire tourner l'entreprise au quotidien, une commande égale une écriture

C'est la même entreprise, réorganisée pour deux usages. Comprendre le passage de l'un à l'autre, c'est le cœur du métier d'ingénieur analytique, et c'est la raison pour laquelle cette formation vous fait travailler les deux à partir du Cours 02.

Le plan : monter AdventureWorks DW dans DuckDB (Partie 2, à faire maintenant), vérifier (Partie 3), puis ajouter PostgreSQL et AdventureWorks OLTP (Partie 4, avant le Cours 02).

Pourquoi DuckDB pour l'entrepôt. Un entrepôt de données n'est pas forcément une plateforme distante : c'est d'abord une façon de ranger les données, en étoile, pour les lire par millions de lignes. DuckDB fait exactement cela, sur un seul fichier, sans serveur, sans compte, sans réseau et sans facture. Le SQL analytique que vous écrirez ici, le fenêtrage, les CTE, les pivots, les jointures as-of, est celui des grandes plateformes : la moitié des requêtes de cette formation passent telles quelles sur Snowflake, sur BigQuery ou sur Databricks.

Partie 2 — Monter AdventureWorks DW dans DuckDB

Trois gestes : créer le dossier de travail, y déposer les fichiers de Microsoft, lancer le script.

2.1 Le dossier de travail

Tout ce que vous montez pour cette formation tient dans un dossier, `Projet_SQL/`, rangé dans le dossier du parcours, `Data_Domain/`, à côté de `Projet_Boreal/`. Ouvrez l'Anaconda Prompt, placez-vous dans `Data_Domain`, puis :

© 2026 iAlexMG. Tous droits réservés.

```
mkdir Projet_SQL
cd Projet_SQL
```

Sauf mention contraire, les commandes de ce module se tapent de là.

2.2 Récupérer les fichiers de données

Sur la page des *releases* du dépôt Microsoft (https://github.com/microsoft/sql-server-samples/releases/tag/adventureworks), téléchargez `AdventureWorksDW-data-warehouse-install-script.zip`, la version entrepôt. Décompressez son contenu directement dans `Projet_SQL/` : vous obtenez une trentaine de fichiers `.csv`, un par table. Le script en lit huit.

Deux faux amis à éviter. Le zip `AdventureWorks-oltp-install-script.zip` est la version OLTP, pour PostgreSQL, Partie 4, pas celle-ci. Et les fichiers `.bak` sont des sauvegardes SQL Server que DuckDB ne sait pas lire. Il vous faut bien le zip DW.

2.3 Lancer le script de montage

Le script vit avec le cours, `Cours_00_Installation/monter_entrepot_dw.sql`. Depuis `Projet_SQL/`, une seule commande :

```
duckdb adventureworksdw.duckdb -f ../SQL/Cours_00_Installation/monter_entrepot_dw.sql
```

Il crée le schéma `dbo`, les huit tables avec leurs vrais noms de colonnes, puis il y verse les fichiers. Il affiche pour finir le compte de lignes de chaque table, que la Partie 3 reprend. Sur le poste de l'auteur, il tourne en une seconde et produit un fichier `adventureworksdw.duckdb` de 25 Mo, pour 35 Mo de fichiers lus.

Ce fichier est l'entrepôt : une base de données entière dans un fichier unique, qu'on copie, qu'on sauvegarde ou qu'on efface comme n'importe quel document. Il n'y a ni service à démarrer, ni port, ni mot de passe.

Pourquoi un script, et pas un chargement à la main. Les fichiers d'AdventureWorks DW n'ont pas de ligne d'en-tête : ils commencent directement par les valeurs. Aucun outil de chargement ne peut donc deviner le nom des colonnes ; il les appellerait `C1`, `C2`, `C3`, et vos requêtes (`SELECT ProductKey…`) échoueraient toutes. La solution, propre et durable, est de définir les tables nous-mêmes, avec les vrais noms et les vrais types, puis de charger par position. C'est du DDL, justement la première famille de commandes du Cours 01, et c'est ce que le script contient : ouvrez-le, il est commenté pour être lu.

`dbo`, pour *database owner*, est le nom de schéma d'origine d'AdventureWorks. On le garde pour coller à la documentation de Microsoft, qui écrit partout `dbo.FactInternetSales`.

2.4 Ouvrir l'entrepôt et poser le contexte

Pour travailler, on ouvre le fichier avec la ligne de commande DuckDB :

```
duckdb adventureworksdw.duckdb
```

Puis, une fois à l'invite, on se place dans le schéma `dbo` :

```
USE adventureworksdw.dbo;
```


© 2026 iAlexMG. Tous droits réservés.

Sans cette ligne, le schéma courant est `main`, qui est vide : `SELECT * FROM DimProduct` répondrait que la table n'existe pas, alors qu'elle est bien là. On peut aussi s'en passer en écrivant le nom complet à chaque fois, `dbo.DimProduct`.

Le nom de la base, lui, n'a pas été choisi : DuckDB le tire du nom du fichier, sans son extension. Renommer `adventureworksdw.duckdb` renomme donc la base, et casse tout ce qui la nomme. Pour cette raison, les labos de la formation écrivent la forme courte, `USE dbo;`, qui désigne le schéma dans la base déjà ouverte et se moque du nom du fichier. C'est leur première ligne.

Piège de casse. DuckDB ne tient pas compte de la casse des identifiants : `dimproduct`, `DimProduct` et `DIMPRODUCT` désignent la même table, et il garde pour l'affichage la casse déclarée à la création. Les supports écrivent donc `DimProduct` et `ListPrice`, la casse d'origine. Le second terrain, lui, ne pardonne pas : PostgreSQL replie tout en minuscules, et c'est le sujet d'un piège du Cours 02.

Pour quitter, `.quit` ou `Ctrl+D`.

Piège. Un fichier DuckDB ouvert en écriture n'accepte qu'un seul processus. Si vous laissez la ligne de commande ouverte sur `adventureworksdw.duckdb` et que vous lancez un script Python dessus, le second reçoit `IO Error: Cannot open file […] utilisé par un autre processus`, avec le nom du programme fautif. Fermez la fenêtre qui tient la base, et relancez.

Partie 3 — Vérifier que l'entrepôt est prêt

Le script a déjà affiché les huit comptes en finissant. Les voici, à retrouver :

Table	Rôle	Lignes	Sert dès le…
`FactInternetSales`	faits : les ventes en ligne, le centre de l'étoile	60 398	Cours 01
`DimProduct`	produit	606	Cours 01
`DimProductSubcategory`	sous-catégorie	37	Cours 01
`DimProductCategory`	catégorie de produit	4	Cours 01
`DimDate`	date et calendrier	3 652	Cours 02
`DimEmployee`	employé (hiérarchie)	296	Cours 02
`DimSalesTerritory`	territoire de vente	11	Cours 06

© 2026 iAlexMG. Tous droits réservés.

Table	Rôle	Lignes	Sert dès le…
DimCustomer	client	18 484	Cours 02

Pour les revoir plus tard, dans la ligne de commande DuckDB :

```
USE dbo;
SHOW TABLES;
SELECT COUNT(*) AS nb_produits FROM dbo.DimProduct;        -- 606
SELECT COUNT(*) AS nb_ventes   FROM dbo.FactInternetSales; -- 60 398
```

Notez les deux derniers nombres : ils vous serviront de repère tout au long de la formation. Si une requête d'un cours ultérieur vous rend un compte très différent sans raison apparente, c'est souvent qu'une jointure a éclaté les lignes. Si ces requêtes répondent, l'essentiel est prêt : direction le Cours 01.

Astuce. Le script se relance sans danger, autant de fois qu'on veut : ses `CREATE OR REPLACE TABLE` refont les tables à neuf avant de recharger, et les comptes sont les mêmes. Une seule conséquence, mesurée : le fichier grossit à chaque passage, de 25 à 39 Mo au second, parce que DuckDB ne rend pas tout de suite l'espace libéré. Pour repartir net, effacez `adventureworksdw.duckdb` avant de relancer.

Partie 4 — PostgreSQL et AdventureWorks OLTP (le second terrain)

À faire avant d'attaquer le Cours 02, où le double chemin commence. Chaque module a dès lors deux laboratoires, un par moteur, et c'est leur comparaison qui fait l'essentiel de la valeur du cours. Rien ne vous empêche de le faire dès maintenant.

4.1 PostgreSQL sur le poste

L'installation de PostgreSQL ne se fait pas ici : elle est à l'étape 7 du guide d'installation du parcours, `Data_Domain/00_Installation/Guide_installation.md`, avec l'installateur d'EDB, le réglage du chemin de recherche et la vérification. Comptez une quinzaine de minutes. À la fin, dans un terminal :

```
psql --version
```

Il répond `psql (PostgreSQL) 18.6`, ou un numéro voisin. Gardez sous la main le mot de passe de l'utilisateur `postgres`, choisi à l'installation : tout ce qui suit le redemande.

Un client pour écrire les requêtes : pgAdmin 4 (livré, simple pour débuter), DBeaver (https://dbeaver.io/) ou `psql` en ligne de commande. Les requêtes sont les mêmes partout. Mais le chargement qui suit passe par `createdb` et `psql`, qui sont des programmes : pgAdmin ne les remplace pas, et le chemin de recherche n'est donc pas facultatif.

4.2 Charger AdventureWorks (OLTP)

© 2026 iAlexMG. Tous droits réservés.

AdventureWorks est née pour SQL Server. On la charge dans PostgreSQL via un portage communautaire. Choisissez une des deux voies. La voie A est celle du parcours ; la voie B évite Ruby si vous avez déjà Docker.

Voie A, avec le portage, qui demande Ruby.

1. Installez Ruby : https://rubyinstaller.org/, le « Devkit » est inutile. Vérifiez avec `ruby --version`.
2. Récupérez le portage : https://github.com/lorint/AdventureWorks-for-Postgres, par *Code*, *Download ZIP*, puis décompressez.
3. Téléchargez le script AdventureWorks 2014 OLTP de Microsoft (même dépôt qu'en Partie 2) et décompressez ses fichiers `.csv` dans le dossier du portage, à côté de `update_csvs.rb`. Laissez de côté le `.sql` de l'archive, écrit pour SQL Server : le portage a le sien, `install.sql`.
4. Ouvrez l'Anaconda Prompt et placez-vous dans le dossier du portage, celui qui contient `install.sql`, `update_csvs.rb` et les `.csv`. Les quatre commandes se tapent de là, dans cet ordre :

```
set PGCLIENTENCODING=UTF8
ruby update_csvs.rb
createdb -U postgres Adventureworks
psql -U postgres -d Adventureworks -f install.sql
```

Ce que chacune fait, et ce que vous devez voir. La première ne répond rien : elle dit à `psql` que les fichiers sont en UTF-8, et elle vaut pour cette fenêtre seulement. La deuxième réécrit les `.csv` au format que PostgreSQL attend, sans rien afficher, en une dizaine de secondes. La troisième crée la base et demande le mot de passe de `postgres` : rien ne s'affiche pendant que vous le tapez, c'est normal, et une création réussie ne dit rien non plus. La quatrième redemande ce mot de passe, puis déroule des centaines de lignes `CREATE TABLE`, `COPY n` et `ALTER TABLE` pendant une à deux minutes. Aucune ligne ne doit commencer par `ERROR`.

Piège — l'encodage des accents, qui ne dit rien quand il se trompe. Le script du portage ne précise l'encodage que sur trois de ses soixante-neuf commandes `\copy` ; pour les soixante-six autres, c'est l'encodage du client qui décide, et une console Windows annonce sa page de codes, 850 sur le poste de l'auteur. Un `François` lu en 850 entre dans la base abîmé, sans le moindre message. D'où la ligne `PGCLIENTENCODING` avant tout le reste. Le contrôle est à la fin de cette partie.

Piège — Sous PowerShell, deux différences. La variable s'écrit `$env:PGCLIENTENCODING = "UTF8"` et non `set`. Et ne créez pas la base avec un `psql -c` qui contient du SQL entre guillemets : ce terminal ne transmet pas les guillemets internes comme `cmd.exe` le fait, et vous obtenez soit une erreur, soit une base `adventureworks` tout en minuscules, que la commande suivante ne trouvera pas. `createdb` reçoit le nom en argument et non en SQL à citer, donc la casse est respectée dans les deux terminaux.

Voie B, avec Docker, sans Ruby. Docker s'installe à l'étape 4 du guide du parcours, dans l'Ubuntu de WSL 2. Récupérez le portage, renommez le zip Microsoft en `adventure_works_2014_OLTP_script.zip` dans le dossier du portage, puis, depuis Ubuntu et depuis ce dossier, `docker compose up -d`. Le conteneur construit l'image, charge la base et publie le port 5432 : `localhost:5432` depuis Windows, base `Adventureworks`, utilisateur `postgres` et mot de passe `postgres`, tous deux écrits en clair dans le `docker-compose.yml` du portage.

© 2026 iAlexMG. Tous droits réservés.

Piège — deux PostgreSQL sur le même port. Si vous avez déjà installé PostgreSQL sur le poste (étape 7 du guide), il écoute déjà sur 5432 et le conteneur refusera de démarrer, sur `port is already allocated`. Choisissez : ou bien la voie A sur le PostgreSQL du poste, ou bien la voie B seule. Les deux ensemble demandent de republier le conteneur sur un autre port, et tous les exemples de la formation supposent le 5432.

Compléter. Le portage retire une colonne dont les cours se servent partout : `linetotal`, le montant d'une ligne de commande. Chez SQL Server, c'est une colonne calculée ; le portage la garde le temps de lire les fichiers, qui la contiennent, puis la supprime à la fin de `install.sql`. On la remet, calculée cette fois par PostgreSQL, avec la formule d'origine :

```
ALTER TABLE sales.salesorderdetail
    ADD COLUMN linetotal numeric
    GENERATED ALWAYS AS (unitprice * (1.0 - unitpricediscount) * orderqty) STORED;
```

Colonne générée — `GENERATED ALWAYS AS (...) STORED` calcule la valeur à chaque écriture de la ligne et la range sur disque avec le reste. On la lit comme n'importe quelle colonne, mais on ne peut pas y écrire soi-même. Sans cette commande, toute requête qui lit `linetotal`, du Cours 04 au Cours 10, échoue avec l'erreur « la colonne d.linetotal n'existe pas ».

Vérifier. Connecté à `Adventureworks` :

```
SELECT COUNT(*) FROM production.product;          -- 504
SELECT COUNT(*) FROM sales.salesorderdetail;      -- 121 317
SELECT ROUND(SUM(linetotal), 2) FROM sales.salesorderdetail;   -- 109846381.40

SELECT COUNT(*) AS nb_tables                      -- 68
FROM information_schema.tables
WHERE table_type = 'BASE TABLE'
  AND table_schema IN ('humanresources', 'person', 'production', 'purchasing', 'sales');

SELECT COUNT(*) AS avec_accent                    -- 2
FROM person.person WHERE firstname ILIKE 'françois';
```

Le dernier est le contrôle de l'encodage : deux personnes se prénomment François dans la base, et le Cours 02 s'en sert. S'il rend `0`, les accents ont été lus dans la mauvaise page de codes ; reprenez le chargement en entier, `PGCLIENTENCODING` posé d'abord, après avoir supprimé la base.

Ces nombres sont exacts, comme ceux de la Partie 3, et vous servent de repère au même titre. Le troisième confirme le retour de `linetotal` : c'est le montant total des ventes, hors taxes et transport, et il tombe au cent près sur le sous-total que la base range dans les en-têtes de commande. Le dernier vérifie que le script est allé au bout : on retrouve les 68 tables du tableau de la Partie 1, réparties dans cinq schémas. S'il donne bien 68 mais qu'un des deux premiers comptes répond zéro, les tables ont été créées sans être remplies : un fichier `.csv` manquait dans le dossier du portage au moment du chargement.

Piège — Si le chargement s'arrête en cours de route, ne relancez pas le script par-dessus : il commence par des `CREATE TABLE`, qui échoueront tous sur les objets déjà créés, et le vrai message d'erreur se noiera dans le flot. Supprimez la base, recréez-la vide, corrigez la cause du premier échec (un fichier au mauvais endroit, neuf fois sur dix), puis relancez en entier, sans oublier l'étape « Compléter ».

© 2026 iAlexMG. Tous droits réservés.

Deux scripts, deux tempéraments. Celui de la Partie 2 se rejoue sans rien casser ; celui-ci, non. La différence n'est pas une question de moteur, c'est une question d'écriture : le premier remet chaque table à neuf avant de la remplir, le second suppose une base vide. C'est la notion d'idempotence, et elle vous suivra bien au-delà du SQL.

Dépannage

Message ou symptôme	Ce qui se passe	Quoi faire
`IO Error: No files found that match the pattern "FactInternetSales.csv"`	le terminal n'est pas dans le dossier des données : les chemins du script sont relatifs au terminal, pas au script	`cd` dans `Projet_SQL/`, là où sont les `.csv`, et relancer
`IO Error: Cannot open file "adventureworksdw.duckdb"`, avec un nom de programme et un numéro	un autre processus tient déjà la base en écriture	fermer la fenêtre ou le programme nommé, puis relancer
`Table with name DimProduct does not exist!` alors que la table est là	le schéma courant est `main`	`USE dbo;`, ou écrire `dbo.DimProduct`
`duckdb` : commande introuvable	la ligne de commande n'est pas installée, ou le terminal a été ouvert avant	étape 3 du guide d'installation, puis rouvrir le terminal
`psql` : commande introuvable	le dossier `bin` de PostgreSQL n'est pas dans le chemin de recherche	étape 7 du guide d'installation, puis rouvrir le terminal

---

Annexe — Le schéma en étoile d'AdventureWorks DW (référence)

Gardez cette carte sous la main : c'est celle de notre base de référence. Au centre, la table de faits `FactInternetSales`, où chaque ligne est une ligne de commande, avec des mesures (quantité, montant) et des clés vers les dimensions. Autour, les dimensions répondent aux questions quoi, quand, qui, où.

Dimension	Question à laquelle elle répond	Clé qui la relie au fait	Ce qu'on y trouve
`DimProduct`	quoi ?	`ProductKey`	nom, couleur, prix de liste, sous-catégorie

© 2026 iAlexMG. Tous droits réservés.

Dimension	Question à laquelle elle répond	Clé qui la relie au fait	Ce qu'on y trouve
`DimDate`	quand ?	`OrderDateKey`	année, trimestre, mois, jour de la semaine
`DimCustomer`	qui ?	`CustomerKey`	nom, sexe, revenu, et une clé de géographie dont la dimension n'est pas montée ici
`DimSalesTerritory`	où ?	`SalesTerritoryKey`	territoire, pays, continent

La chaîne produit se prolonge au-delà de la première dimension, en s'éloignant du centre : `FactInternetSales` → `DimProduct` → `DimProductSubcategory` → `DimProductCategory`. Une dimension qui en appelle une autre, c'est ce qui distingue un flocon d'une étoile stricte, et notre entrepôt en porte plusieurs cas, pas seulement celui-là. `DimEmployee` a une clé de territoire, qui mène à `DimSalesTerritory`, et une clé vers son propre supérieur, qui replie la hiérarchie des employés dans la table elle-même. `DimCustomer`, enfin, porte une clé de géographie dont la dimension n'est pas montée par notre script : la colonne est là, la table au bout n'y est pas. Celui de la chaîne produit reste le plus net, et c'est celui que le Cours 01 fera joindre.

Analyser les ventes, c'est les croiser avec ces dimensions : tout l'art de l'OLAP tient dans ce geste, et vous ne ferez pas grand-chose d'autre d'un bout à l'autre de la formation.

Et ensuite ?

Direction le Cours 01, qui pose les fondations du SQL sur cette base. Les exercices de ce module, dans `Exercices_corriges_00_SQL.md`, vérifient que vous avez compris ce que vous venez de monter : faites-les avant de passer à la suite. Comptez une heure et demie, chronométrée sur un lecteur débutant : ils vous éviteront des heures de confusion plus tard.

Ce module ne se refait pas. Le jour où vous voudrez reprendre la formation depuis le début, une commande lancée depuis `Data_Domain` ramène l'entrepôt et la base à l'état où vous les laissez ici : `python _univers/remise/remettre_a_zero.py --formation SQL`. Avec `--cours 08`, elle ne remet à neuf qu'un cours. Sans `--go`, elle montre seulement ce qu'elle ferait ; avec, ce qu'elle retire part dans une archive datée de `000-OLD/`, et rien ne se perd.

© 2026 iAlexMG. Tous droits réservés.